Supplementary material: complete proofs

The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes

Jian Gao (高健)

This supplement gives complete written proofs of the results of §6.5, the end of §7.3, §8.5 and §9, together with the original comb/CPS1 installations of §10.1. The main text states the results and gives proof ideas; here each result is given its objects and domain, actual inputs and quantifiers, key lemmas, decisive derivation, and the direct use of the same object in later results, and is finally matched to its formal proof. Notation is that of the main text, and numbers of definitions, theorems and equations refer to the main text. The proofs of §2–§5, §6.1–§6.4, §7.1–§7.3 and §8.1–§8.4 are complete in the main text and Appendices A–C and are not repeated.

The "formal correspondence" at the end of each section lists the Lean declarations that carry the result; the source commit and full path are in Appendices D.2–D.3 of the main text, and the public reproduction entry is in its "Code and data availability".

Fixed sources of the added proofs: PR = `e05558097256e86b537c019c8ca63cd449d6b2d2`, MF = `c62f3d25c42d4bf2b77ac009b49fc71cfd752f46`, MV2 = `71e94e8261d67cf4849a05eb14ac95aa2f7dcbb9`, MV3 = `9c73a630ce05bea062ec889b188f5105c0afb796` (Homework); H0-S = `a1adcc9f4a06b872c381b8ad49513d6bf1cd621a` (public source configuration). Appendix D.3 expands path abbreviations and lists declaration entry points.

S0 Common tools

S0.1 Remaining count and execution. Proposition 3.7 gives: every execution step decreases the remaining count by exactly 1; constants have no execution step; the left-to-right execution $\mathrm{exec}_\rho(e) : e \to \mathrm{const}(\mathrm{ev}_\rho e)$ has length $r(e)$; every execution trace is a derivation, so its endpoints evaluate equally (Proposition 3.6(1), below called **soundness**).

S0.2 The pair language preserves the remaining count. The pairing $e \mapsto \widetilde{e}$ of Appendix A.2 replaces variables, constants, addition, linear and bilinear nodes one by one by nodes of the same kind on pair values, so $r(\widetilde{e}) = r(e)$ (structural induction on e; every node contributes the same count), and

$$\mathrm{ev}_{(\rho,\varepsilon)}(\widetilde{e}) = (\mathrm{ev}_\rho(e),\ \mathrm{eff}_{\rho,\varepsilon}(e)). \tag{S0.1}$$

S0.3 Relation image of a trace and the updated reading. Let $t : e \to e'$ be an execution trace in environment ρ. Every execution step is a derivation certificate whose relation image is the

difference of its endpoints; the relation image of the whole trace telescopes to $e-e'$. By soundness, $e-e' \in \ker(\mathrm{ev}_\rho)$. Given an increment ε, the update equation of Proposition 3.3 gives

$$\mathrm{ev}_{\rho+\varepsilon}(e-e') = \mathrm{ev}_\rho(e-e') + \mathrm{eff}_{\rho,\varepsilon}(e-e') = \mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e'). \tag{S0.2}$$

In particular, when $e' = \mathrm{const}(c)$ we have $\mathrm{eff}(e') = 0$, and the updated value is $\mathrm{eff}_{\rho,\varepsilon}(e)$.

S0.4 Coimages and induced maps. For a $\mathbb{Z}$-linear map $f : X \to Y$, write $\mathrm{Res}(f) := X/\ker f$ for its coimage and $\mathrm{can}_f : X \to \mathrm{Res}(f)$ for the quotient map; $\mathrm{Res}(f) \cong \mathrm{range}(f)$. If $f' \circ s = t \circ f$ (call (s,t) a morphism from f to f'), then $s(\ker f) \subseteq \ker f'$, so s induces $\overline{s} : \mathrm{Res}(f) \to \mathrm{Res}(f')$ with $\overline{s} \circ \mathrm{can}_f = \mathrm{can}_{f'} \circ s$. Moreover $\mathrm{can}_f(x) = 0$ iff $f(x) = 0$.

S0.5 Lifting residual. Let (s,t) be a morphism from $f : C \to D$ to $f' : C' \to D'$. For $c \in C$ its **fibre** is $\mathrm{Fib}_f(c) := \{x : f(x) = f(c)\}$. The map s sends $\mathrm{Fib}_f(c)$ into $\mathrm{Fib}_{f'}(sc)$ and $\ker f$ into $\ker f'$. For $y \in \mathrm{Fib}_{f'}(sc)$ put

$$\mathrm{lift}(c,y) := [\, y - s(c) \,] \in \ker f' \big/ s(\ker f).$$

Lemma S0.5. $\mathrm{lift}(c,y) = 0$ iff there is $x \in \mathrm{Fib}_f(c)$ with $s(x) = y$.

Proof. If $y - s(c) = s(k)$ with $k \in \ker f$, take $x = c + k$; then $f(x) = f(c)$ and $s(x) = y$. Conversely, if $x \in \mathrm{Fib}_f(c)$ and $s(x) = y$, then $k = x - c \in \ker f$ and $y - s(c) = s(k)$. ∎

S0.6 Action-word model. Let X be an additive group with a family $(A_\ell)_{\ell \in L}$ of additive actions and an additive read-out $\mathrm{rd} : X \to Y$. For a word $w = \ell_1 \cdots \ell_n$ (acting first by ℓ_1) write $A_w := A_{\ell_n} \cdots A_{\ell_1}$. Let $K := \{x : \forall w,\ \mathrm{rd}(A_w x) = 0\}$.

Lemma S0.6. (i) K is closed under every A_ℓ, so every A_ℓ descends to $\mathrm{Mod} := X/K$; (ii) $[x] = [y]$ iff $\forall w,\ \mathrm{rd}(A_w x) = \mathrm{rd}(A_w y)$; (iii) the map sending $[x]$ to the family of read-outs $(w \mapsto \mathrm{rd}(A_w x))$ is injective, so Mod is linearly isomorphic to the image of this family (the completion carrier of all word read-outs), and the isomorphism commutes with every action.

Proof. (i) If $x \in K$ then $\mathrm{rd}(A_w A_\ell x) = \mathrm{rd}(A_{\ell w} x) = 0$ for every w. (ii) By the definition of K and linearity. (iii) By (ii); commutation is checked word by word on representatives. ∎

S0.7 Effect-history equation. Let also (U_ℓ) be additive actions on W and $M : X \to W$ an additive observation. For a point x and a word w let the total residual be $R(w,x) := U_w M(x) - M(A_w x)$ and the single-letter residual $r(\ell,x) := U_\ell M(x) - M(A_\ell x)$; define recursively

$$\mathrm{Acc}([\,], x) = 0, \qquad \mathrm{Acc}(\ell w, x) = U_w\, r(\ell, x) + \mathrm{Acc}(w, A_\ell x).$$

Lemma S0.7. $\mathrm{Acc}(w,x) = R(w,x)$; the history recording $(\ell_k, x_{k-1}, r(\ell_k, x_{k-1}))$ has length $|w|$.

Proof. Induction on the length of w:

$$\begin{aligned} R(\ell w, x) &= U_w U_\ell M(x) - M(A_w A_\ell x) \\ &= U_w(U_\ell M(x) - M(A_\ell x)) + (U_w M(A_\ell x) - M(A_w A_\ell x)); \end{aligned}$$

the first term is $U_w\, r(\ell, x)$ and the second is $R(w, A_\ell x)$. ■

S0.8 Joint action-word carrier. Let I, H, Q be additive groups with three families of actions $(\alpha_\ell), (\eta_\ell), (\mu_\ell)$ and two additive read-outs $c : I \to H$, $m : I \to Q$. On $I \times H \times Q$ put $\theta_\ell := \alpha_\ell \times \eta_\ell \times \mu_\ell$ and $\mathrm{seed}(i) := (i, c(i), m(i))$. The **complete orbit carrier** $\mathcal{C}$ is the submodule spanned by all $\theta_w(\mathrm{seed}\, i)$.

Lemma S0.8. (i) $\mathcal{C}$ is the smallest submodule containing all seeds and closed under every θ_ℓ; (ii) the three projections π_I, π_H, π_Q commute with the actions of words; (iii) the incidence element $\mathrm{inc}(w, i) := \theta_w(\mathrm{seed}\, i) - \mathrm{seed}(\alpha_w i)$ has I-component zero, and its H- and Q-components are $\eta_w c(i) - c(\alpha_w i)$ and $\mu_w m(i) - m(\alpha_w i)$.

Proof. (i) The generated submodule contains the seeds by definition; any submodule containing the seeds and closed under the actions contains every $\theta_w(\mathrm{seed}\, i)$ and hence their span. $\mathcal{C}$ itself is closed because the image of a generator is again a generator. (ii) θ_ℓ is a product. (iii) Componentwise from (ii). ■

S1 Theorem 6.10: birth and payment of a responsibility without a prior bearer

Objects and domain. Fix a world network N, an authoritative root on it with complete whole-ledger registration, an origin current c, and a reader that gives a complete environment–expression pair at every occurrence of the root at c. Let o be the canonical occurrence at c, (ρ, e) the output of the reader at o, $B = r(e)$, and $\Lambda = \Lambda_{\rho,e}$ the execution debt law of Definition 6.9.

Inputs and quantifiers. The theorem holds for all such data. No old responsibility row, registered inquiry, program table, table of future requests or empty-coverage certificate is supplied; the origin support stays fixed during the whole calculation.

Construction.

1. *Extended world and vocabulary.* The extended world $N[\Lambda]$ is given by Definition 6.3: its supports are Sup × Option(debt states). The currents of the vocabulary are the debt states of Λ, i.e., pairs (e', t) with $t : e \to e'$ a paid execution trace; the initial current is (e, id). The support of the current x is $(\mathrm{supp}(o), \mathrm{some}\ x)$.
2. *Event algebra.* At the current $x = (e', t)$ the **source-selected action** is the output of the executor: a settlement if e' is a constant, and otherwise the first step $s : e' \to e''$ of the execution of Proposition 3.7. The successor current x^+ is x at a settlement and $(e'', t \cdot s)$ at an execution step. Compilation sends a settlement to the identity transport of Λ (fourth item of Definition 6.9) and an execution step to a native write.
3. *Whole-ledger evolution.* $\mathrm{whole}(x)$ sends every row on the support $(\mathrm{supp}(o), x)$ to $(\mathrm{supp}(o), x^+)$: the mathematical row $\mathrm{row}(x)$ (the debt row of Λ at state x) goes to $\mathrm{row}(x^+)$, and every old row is transported unchanged. Destination and origin are mutually inverse.
4. *Patch, remainder and certificate.* The finite patch contains exactly one generated row, the mathematical row; all other rows are covered by the same-source transported remainder, whose occurrence is the evidence "this row belongs to the old ledger"—no finiteness of the old ledger is needed. The patch folds back to the whole-ledger evolution: $\mathrm{fold}(\mathrm{patch}(x)) = \mathrm{whole}(x)$. The

restructuring certificate is generated by injectivity of the destination map of the whole-ledger evolution.

5. *Compiler, root and process.* The compiler emits patch and whole ledger at every current; the ledger root consists of source and compiler; the authoritative root and the living process are obtained by the standard construction of root registration. The state of the process at count k is written x_k, with $x_0 = (e, \mathrm{id})$ and $x_{k+1} = x_k^+$.

Key lemmas.

Lemma S1.1 (one-step budget). $r(x^+) = r(x) - 1$ (natural subtraction); if $r(x) = 0$ then $x^+ = x$.

Proof. If $r(x) > 0$ then e' is not a constant, the source-selected action is an execution step, and by S0.1 the count drops by one; if $r(x) = 0$ then e' is a constant, the action is a settlement and the successor is unchanged. ■

Lemma S1.2 (budget and history). For every k: $r(x_k) = B - k$ and $|t_k| + r(x_k) = B$, where t_k is the paid trace of x_k.

Proof. Induction on k. For $k = 0$, t_0 is empty and $r(x_0) = B$. Induction step: if $r(x_k) > 0$, the execution step lengthens the trace by one and decreases the count by one; if $r(x_k) = 0$, the state is unchanged and $B - (k+1) = 0$. ■

Decisive derivation.

1. *Birth.* By construction steps 2–4, every current emits exactly one event; compilation is the identity transport at a settlement and a native write at an execution step. The patch contains exactly one generated row; all old rows are kept as the remainder, and mutual inversion of origin and destination gives the restructuring certificate. The original support is unchanged (the first component of the support is always $\mathrm{supp}(o)$), and the law surface, old projections and type and value observations at the origin are kept as root readings.
2. *Payment.* The same-debt current at visit k is $\mathrm{row}(x_k)$; the destination equation $\mathrm{whole}(x_k)(\mathrm{row}(x_k)) = \mathrm{row}(x_{k+1})$ gives by induction on k its lineage to $\mathrm{row}(x_0)$. Its budget is $r(x_k) = B - k$ (Lemma S1.2). For $k < B$, $r(x_k) > 0$, the source-selected action is an execution step s, the same-debt target is $\mathrm{row}(x_{k+1})$, and the budget drops strictly (Lemma S1.1); composed with empty histories on both sides it is a payment macro of Definition 6.6. The target budget is at most the current budget, so there is no refill; the continuation relation strictly decreases the budget and is well founded.
3. *Settlement.* At $k = B$ we have $r(x_B) = 0$, so the expression of x_B is a constant $\mathrm{const}(v)$ and a local settlement exists; a current of budget zero has no payment macro. The paid trace $t_B : e \to \mathrm{const}(v)$ has length B (Lemma S1.2), and soundness gives $v = \mathrm{ev}_\rho(e)$. By Lemma S1.1, $x_{B+1} = x_B$; the visit count keeps advancing, and visits of different causal depth are still distinguished in finite histories.
4. *Receipt.* By S0.3 the relation image of t_B is $e - \mathrm{const}(\mathrm{ev}_\rho e) \in \ker(\mathrm{ev}_\rho)$; for every increment ε, its value under $\rho + \varepsilon$ is $\mathrm{eff}_{\rho,\varepsilon}(e)$. ■

Direct use. Proposition 6.11 applies this theorem with the whole-tree program as reader; Theorem 7.7 and every run of §9 use the same owner-free installation as their mathematical row, reading its value, charged history, original material and relation boundary.

Formal correspondence. `JS/OwnerFree/Source.lean` (`vocabulary`, `eventAlgebra`, `whole`, `mathEntry`, `destination_math`); `OwnerFree/Compiler.lean` (`remainder`, `patch`, `patch_fold`, `compiler`, `ledgerRoot`); `OwnerFree/Completion.lean` (`next_budget`, `budget`, `history_accounting`, `completed_value`, `completed_history_length`, `completed_next_state`); `OwnerFree/Payment.lean` (`lineage`, `debtCurrent`, `activePayment`, `wellFounded`, `no_refill`, `endpoint_no_paid`); `OwnerFree/Consumer.lean` (`value_source`, `paid_history`, `relation_boundary`, `updated_inverse_fibre`). Commit AB.

S1.3 Theorem 6.12: Type completion receipts and continuing births

Data and construction. Fix the original programme $\mathcal{P}$, recursive frames Φ_i and actual actions a_i, with B_i the remaining count of the current paid-state expression. The receipt takes an actual starting point i and returns the dependent sum

$$\sum_{d:\mathbb{N}} \{s : \mathsf{Settlement}(\Phi_{i+d}.\mathrm{state}) \mid d \le B_i \ \wedge \ a_{i+d} = \mathrm{inl}\, s\}.$$

Use strong recursion on B_i and eliminate a_i. A settled branch returns $d = 0$ and its actual s. A paid branch has $a_i = \mathrm{inr}\, p$, whose target **is** the mathematical state of Φ_{i+1}. Since $B_{i+1} = B_i - 1 < B_i$, read the receipt $(d', s', ...)$ at that exact start and return $(d' + 1, s', ...)$. Equality $(i + 1) + d' = i + (d' + 1)$ transports the settlement event; $d' \le B_i - 1$ gives $d' + 1 \le B_i$. This defines a Type receipt at every start, retaining each actual branch equality.

Distance equals budget. Induct on $h \le B_i$. For $h < B_i$, the induction gives $B_{i+h} = B_i - h > 0$. Settlement requires a constant expression and zero budget, so the actual action must be paid. Its actual next frame has $B_{i+h+1} = B_i - h - 1$. Consequently

$$B_{i+h} = B_i - h \quad (0 \le h \le B_i), \qquad a_{i+h} = \mathrm{inr}\, p_h \quad (h < B_i).$$

The receipt endpoint is actually settled, hence $B_{i+d} = 0$. Its bound $d \le B_i$ allows substitution in this formula, yielding $d = B_i$. In particular, a zero-budget start returns distance zero without manufacturing a payment or bypassing settlement.

Complete state and environment. Each p_h appends an actual execution step to the original Trace and preserves the registered input. Induction gives the exact paid prefix and every intermediate state. Soundness gives endpoint constant value $\mathrm{ev}_\rho e$. The original environment reader's registered compatibility law reads back the same environment during mathematical advance with this fixed input. Apply that law successively along the actual payment equalities: the environment is preserved for $0 \le h \le B_i$. Neither budget nor distance uses this law.

Original compilation and birth. Expand the original compiler at the actual endpoint equality $a_{i+B_i} = \mathrm{inl}\, s$. Settlement invokes original debt admission and emits the active presentation of $\mathrm{nextBorn}(\Phi_{i+B_i}, \mathcal{P})$ as canonical next. The sealed run's `actual_node` identifies this with its original frame at $i + B_i + 1$. The admission successor and original next frame are thus fully equal.

Finally set $u_0 = i$, $v_j = u_j + B_{u_j}$ and $u_{j+1} = v_j + 1$. Every v_j carries its own Type receipt above and $v_{j+1} \ge v_j + 1$. Induction gives $v_j \ge i + j$; for any original tick t, take $j \ge t$ to obtain $t \le v_j$. The birth sequence is cofinal. Each segment $u_j, \dots, v_j$ still consists of values of the original recursive frame function at those indices. ■

Direct consumers and fixed source. Total-value birth environment uses the actual settlement stored in the receipt (S5.2); original packet stops attach every complete paid prefix to the same run (S5.3). `ACT/Completion.lean`, `Completion/Prefix.lean` and `Completion/Births.lean`, including `receipt`, `receipt_distance_exact`, `receipt_next`, `receipt_compiles`, all prefix and birth-cover declarations, originate at MV2; their original constructions and consumers are read at MV3.

S2 Theorem 8.12 and Theorem 7.7

S2.1 Theorem 8.12: the actual action generates the next question

Objects. An inquiry frame Φ: an inquiry state, the complete native program reading the root compiler, the raw input (ρ, e) registered at the current visit, a reader E giving the actual environment at every original occurrence, and a causal depth k. The mathematical state of the frame is a paid state (e', t) of $\Lambda_{\rho,e}$; ρ' is the reading of E at the frame's occurrence and $\varepsilon = \rho' - \rho$.

(1) Value of the request. The residual request $e \ominus e' = e + \mathrm{linear}_{-\mathrm{id}}(e')$ is registered on $\rho + \varepsilon = \rho'$. By the definition of evaluation $\mathrm{ev}_{\rho'}(e \ominus e') = \mathrm{ev}_{\rho'}(e) - \mathrm{ev}_{\rho'}(e')$. Soundness gives $\mathrm{ev}_{\rho}(e) = \mathrm{ev}_{\rho}(e')$, so the old value is zero. The relation image of the paid trace is $e - e'$ (S0.3), whose value under ρ' is by (S0.2) $\mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e')$, in agreement with the request's value. By S0.4 the canonical residual of this certificate under $\mathrm{ev}_{\rho'}$ vanishes iff its value vanishes, i.e., iff the request's value vanishes. At a settlement $e' = \mathrm{const}(\mathrm{ev}_{\rho} e)$ and the request's value is $\mathrm{eff}_{\rho,\varepsilon}(e)$.

(2) Necessary birth. $r(e \ominus e') = r(e) + (r(e') + 1) + 1$: the addition node contributes 1 and the linear node 1. This is positive, so the initial state is not a constant and the executor's first action is an execution step; in the birth compilation this step compiles to an obligation admission (the birth of a new responsibility), not to a payment of any old debt row.

(3) Recursive frames and run. Define the **rank** of a frame $\mathrm{rk}(\Phi) := (r(e),\ k+1)$, ordered lexicographically. The next frame is determined by the current action:

- an execution step gives the same frame with depth increased by one, of rank $(r(e), k+2)$;
- a settlement gives the born frame—old state the current inquiry state, program the successor program, registered input the residual request, depth zero—of rank $(r(e) + r(e') + 2,\ 1)$.

Both cases strictly increase the rank. The sequence of frames Φ_n (Φ_{n+1} the next frame of Φ_n) is therefore strictly increasing lexicographically. The erasure of a frame determines its rank: the raw input is read back by the root's observation (erasure keeps vocabulary and visit, and the observation reads back the registered raw input), and the causal depth is read from the history of the erased visit. If Φ_m and Φ_n had the same erasure with $m < n$, they would have the same rank, contradicting strict increase. So erasure is injective. Validity of successors is checked branch by branch: an execution step uses the successor law of a direct answer, a settlement uses the successor law of an obligation admission, and both preserve the generated living law on the same root. The node of the existing canonical inquiry runtime at count n is the presentation of Φ_n: by induction on n, the activation successor of a node is the next frame. Every frame has a singleton query fibre (its only query is the frame's registered input), so no external table of future requests is needed.

(4) Separation of payment and birth. Restrict actual nodes to the debt process of the fixed root: the execution-step branch emits a same-debt payment receipt on the same mathematical row, whose whole-ledger destination equals the ledger evolution of the joint execution step (the reading of Theorem 6.10(2)); there is no payment macro where the budget is zero (Theorem 6.10(3)); the birth of the settlement branch is a new obligation admission. ■

Formal correspondence. `JS/Native/ResidualRequest.lean` (`expression_eval`, `old_value`, `budget`, `action_is_paid`, `updated_value`, `residual_zero_iff`); in `JS/Native/Restructuring/Inquiry/Continuation/`: `Policy.lean` (`rank`, `Before`, `next_progress`, `frames_forward`), `Inventory.lean` (`rank_eq_of_erasure`, `frames_erase_injective`), `Occurrence.lean` (`successor_valid`), `Runtime.lean` (`actual_node`, `macro_next`, `macro_next_preserves`), `Payment.lean` (`actual_paid_receipt`, `no_paid_of_zero`). Commit AB.

S2.2 Theorem 7.7: shared run of the registered input and successor feed

Objects. The fixed data of Proposition 7.6: an old inquiry state (root, visit, audit protocol, calculus, query type, bearers and authority), a recognition installed on a complete complex inner-product space $\mathcal{H}$, the step at this visit and its disposition settle; in addition, the original inquiry input ι registered by the root at this visit: a projection position, its activation proof at this occurrence, a classifier equation and a query-type equation.

(1) Registered activation. Split by the output of settle.

All-generated or joint-residual branch. Proposition 7.6 gives a program package and the calculation inquiry state, whose root is the root with the calculation inquiry installed. On this root add a **query projection law**: the projection type is a singleton; at an occurrence o' it is active iff o' is the exact occurrence of this visit; the payload type is the calculation query; the projected value is the calculation query whose incidence is the original query of ι. The new root r_ι is the original root with this projection added; the visit is unchanged. The registered input ι' takes this projection, the activation proof at the exact occurrence and the classifier equation (at the exact occurrence the classification takes the active branch by definition); the query-type equation holds by definition. The inquiry state s_ι has root r_ι, keeps visit, audit protocol and calculus of the old state; bearers are read through the query's incidence; authority is the old authority lifted along the new projection face; the compilation program answers every query with the result face of the package embedded through the projection.

So s_ι compiles the query of ι' to an answer (by definition). The expression read by the answer face is the whole-tree program of the package, so the parser reads T back (Theorem 3.9(3) and the tree equation of Proposition 7.6); the value is $\delta_{\tau(T)}$ (Theorem 3.9(1)); the source state is the paid state of the package. The compilation face of the old state for every original query q is read back through the same embedding and still reads the canonical token of the old compilation.

The other four branches. The terminal branch returns the whole-ledger compilation of the root compiler at the source occurrence, and the three residual branches return the residual object itself, as in Proposition 7.6.

(2) Shared run. With s_ι as inquiry state and the constant function "return the raw input of the query of ι'" as reader, construct the mathematical inquiry runtime (the owner-free installation

of Theorem 6.10 plus the runtime of §8.5). The raw input of the owner-free installation is the raw input of this query; by Theorem 3.9(3) its expression reads back T. Theorem 6.10(3) gives a charged history of length $r(\mathrm{prog}_{\kappa}T)$, rewritten to $\beta(T)$ by Theorem 3.9(2), and the completed value $\mathrm{ev}(\mathrm{prog}_{\kappa}T) = \delta_{\tau(T)}$. The whole-ledger write-back and original material of the mathematical row at every count, and the actual query, answer and successor at every runtime offset, are the node equations of the mathematical inquiry runtime at that count and offset (Theorem 8.12(3)).

(3) Successor feed. The inputs are only the living root, the visit v, the recognition and the audit protocol. On nonterminal branches the step carries a successor succ and the equation $\mathrm{next}(v) = \mathrm{succ.current}$; put $v^{+} := v.\mathrm{next}(\mathrm{succ})$. Recognition generates a step at v^{+} and settle classifies it; the constructions of (1)(2) give again a feed and a run at v^{+}. The terminal branch returns only the whole-ledger compilation. Every nonterminal branch returns the preceding program package (the two aligned branches) or residual (the three residual branches) paired with the next run.

Whole-ledger write-backs and next currents of the two steps: the definition of a step contains $\mathrm{whole}(\mathrm{step}(v)) = \mathrm{generatedLedgerAt}(v.\mathrm{current})$ and $\mathrm{nextCurrent}(\mathrm{step}(v)) = \mathrm{generatedNextCurrentAt}(v)$, and the same holds at v^{+}. The successor's target occurrence equals the root's canonical occurrence at the current of v^{+} (the target occurrence is the event the compiler emits for the next current), and history is a function of the occurrence, so the two histories are equal. ■

Direct use. The joint program of §9 (Theorem 9.2) takes the same recognition and the same successor as input; the default run of Theorem 9.8 consumes the next run of (3) at every visit.

Formal correspondence. `DJ/Input/{Source,Consumer}.lean`, `DJ/Input/Runtime/{Source,Consumer}.lean`, `DJ/Next/{Source,Consumer}.lean`; low source inquiry `JS/OwnerFree/Installation/Math/{Inquiry,Frame}/Source.lean`. Commit AC.

S2.3 Theorem 8.14: same-occurrence differences, independent charge and the source environment square

Two actual boundaries. Take the next packet generated by the original source family, then its actual grandnext occurrence. b_{Ψ} is the pair lift of the original terminal execution and b_{μ} the complete boundary of original next-compilation execution. Their respective Traces supply membership in the original next history's relation closure. The occurrence is selected by original generation. In the original after scope let r be the original request point, Ψ its terminal value, $\mu = (\Psi, \sigma)$ its write value, and c the constant embedding. The original compiler's affine boundary formulas are

$$Q^{+}(b_{\Psi}) = r - c(\Psi, 0), \qquad Q^{+}(b_{\mu}) = r - c(\mu).$$

Thus $d = b_{\Psi} - b_{\mu}$ belongs to the same relation closure. Linearity and group cancellation give

$$Q^{+}(d) = (r - c(\Psi, 0)) - (r - c(\mu)) = c(\mu - (\Psi, 0)) = c(0, \sigma).$$

The actual environment of the after scope reads this constant, so the independent difference query evaluates to $(0, \sigma)$. The history closure preserves the responsibility origins of both original

executions; after scope reads the updated value, without assuming that an earlier evaluation kernel remains the updated kernel.

The coefficient programme's own charge. Write d's finite support in its original support-list order as $(e_1, a_1), \dots, (e_k, a_k)$. Define $N_0(e) = \mathrm{const}(0)$ and $N_{n+1}(e) = e + N_n(e)$. A nonnegative integer coefficient expands to $N_a(e)$; a negative coefficient $a = -(n+1)$ expands to $-N_{n+1}(e)$, using one original negation node. Induction on n gives

$$\mathrm{ev}_\rho N_n(e) = n\,\mathrm{ev}_\rho e, \qquad r(N_n(e)) = n(r(e)+1).$$

Integer expansion $E_a(e)$ therefore satisfies

$$\mathrm{ev}_\rho E_a(e) = a\,\mathrm{ev}_\rho e, \qquad r(E_a(e)) = |a|(r(e)+1) + \mathbf{1}_{a<0}.$$

Put $F([]) = \mathrm{const}(0)$ and $F((e,a) :: L) = E_a(e) + F(L)$. List induction gives

$$\mathrm{coeff}(d) = F(\mathrm{supportList}(d)), \qquad c(d) = \sum_{(e,a)\in\mathrm{supportList}(d)} (|a|(r(e)+1) + \mathbf{1}_{a<0} + 1),$$

and $\mathrm{ev}_\rho(\mathrm{coeff}(d)) = \mathrm{ev}_\rho d$, $r(\mathrm{coeff}(d)) = c(d)$. An empty difference has charge zero; the later residual birth has its own positive budget. Theorem 6.10 executes this original coefficient programme, giving an actual Trace of length $c(d)$, its $c(d)+1$ states and its own result $(0, \sigma)$, without consuming another calculation's charge certificate.

Installation and actual next. Register this Raw, result, Trace and all offset material at a projection of the original root. Actual query incidence identifies that registration. Its original answered compilation retains the query result, then residual-birth compilation generates debt admission at the same root and visit. The admission projection returns the material just registered; the complete compiler's remainder inherits old projections. Effect, inverse residual, charge, Trace, root, whole ledger and literal next therefore follow one compilation path. The exact successor supplies the next Raw at the original $k+2$ packet.

Feedback and the complete joint inventory. Let q be this source family's actual full query, q' its paid endpoint, $h = q \ominus q'$, β its Trace boundary and θ the original action binding. Soundness gives $\mathrm{ev}_\rho h = 0$. Substitution evaluation and the update identity give

$$\mathrm{ev}_\rho(h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(\beta), \qquad \mathrm{ev}_\rho(h + h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(\beta).$$

Here $\rho\circ\theta$ denotes the environment obtained by evaluating the original binding in ρ. Since $r(h) \geq 2$ and $r(h + h[\theta]) \geq 3$, this full feedback actually pays; its completed Trace has its own remaining-count length. S0.3 identifies the same value with the updated canonical residual of β. Ordered mixed effects give the same complete result by the stepwise telescoping identity of S0.7. The inventory is

$$\mathrm{seed}(H_{\mathrm{old}}, \mathrm{seed}(\mathrm{seed}(H_{\mathrm{dispatch}}, \mathrm{exposure}(T_q)), \mathrm{exposure}(T_{h+h[\theta]}))).$$

Left and right seed inclusions retain the old inventory, every paid query event and every newly paid feedback event. The original stock-to-joint interface supplies their membership in actual jointStock. The outer reader, inner query and whole-next are the original consumers reading this inventory.

Finite Incoming and the environment square. The actual original configuration's calculation reader is Incoming(Φ, D), with D the original query datum. If pairInventory is empty, it returns D.reader(o). Otherwise it selects an original inventory expression or coefficient expression g of a relation word and returns

$$(D.\text{reader}(o).\text{environment},\ D.\text{reader}(o).\text{expression} + g).$$

Both branches have, by definition, the original query environment. Default actionReader uses `calculationReader.getD(queryReader)`: absent configuration gives the same reader; configured Incoming gives equality by these two branches. The actual source family thus generates the generic `sourceActionEnvironment` contract at every occurrence. Incoming never calls a previous calculationReader, so the new g is added once. Configuration replaces calculationReader alone, preserving original queryReader, environment decoder, both inventory callbacks and receiver/seed.

The actual residual receiver has budget $B = r(e) + r(e') + 2 \geq 2$ and budget $B - 1 \geq 1$ after its first execution step. Hence it is actually paid. This proof uses source residual and exact first-step decrease rather than matching names of query and calculation charges. The source square then aligns actualNativeEnvironment with original query environment, completing the environment chain through original write, Born and next query. ■

Fixed source. The difference and installation originate at PR; joint-carrier writeback at MF. Sources are `Cx/Profile/Finite/Reader/Difference/{Source,Inquiry/Source,Inquiry/Installation,Inquiry/Input,Inquiry/Receipt,Inquiry/Compiler,Inquiry/Actual,Actual}.lean`, `Profile/Producer{,/Consumer}.lean`, `Mixed/{Effect,Receipt,Consumer}.lean`, `Difference/Actual.lean` and `Psi/TerminalTransport.lean`. Public source-configuration repairs are H0's `EffectInstalled.lean`, `GeneratedEnvironment.lean`, `FiniteCalculationInstallation.lean`, `FiniteIncoming.lean`, `ContextualFactory.lean`, `ActualNativeEnvironment.lean` and `EnvironmentErasure.lean`, read at H0-S. Historical original objects and public repairs retain their respective proof origins.

S3 Theorems 9.2 and 9.3

S3.1 Theorem 9.2: one program carries the complete time, the common history and the action difference

Objects. A living root, a temporal visit v, a recognition and an audit protocol; step(v) lies in the all-generated or joint-residual branch, with successor v^+, history transition and pairing alignment; T is the aligned constructor tree of Proposition 7.6.

(1) Temporal codes. A temporal visit of the root is given by one of three inductive histories: finitely reachable from the initial current, sealed cofinal, or reachable after cofinality. For each, encoding and decoding between the inductive types Hist, Post and the original reachability evidence are mutually inverse, defined and verified constructor by constructor. A temporal code is $c = (\text{current}, \text{history code})$; enc and dec take these encodings and decodings componentwise, so they are mutually inverse and enc is injective. The first component of $\text{act}(c) = (\text{rcpt}(c), \text{nx}(c))$ contains c itself, so act is injective. Successor: the successor of a temporal visit is given by the

compiler's next current together with the extension of the history, and the encoding commutes with the extension constructor by constructor, so $\mathrm{enc}(v^+) = \mathrm{nx}(\mathrm{enc}\, v)$.

(2) Joint program. Nodes are pairs (aligned node of T, temporal code), and T_J gives every node of T the code $\mathrm{enc}(v)$. Results are five-tuples

$$(\text{effect fold},\ \text{receipt tree},\ \text{original material of both histories},\\ \text{evaluator inputs of both sides},\ \text{exposure plans of both sides}).$$

The constructor $\kappa_J(n, \vec{y})$: its first component is the single-node fold of the original dependent effect $\mathrm{effectFold}(n_1, [\, y_{i,1} \,])$; its second component is the tree with root $\mathrm{act}(n_2)$ and the receipt trees of the children as subtrees; the last three components are the original material of both histories, the evaluator inputs of both sides and the exposures of both sides, independent of the children. The program is $P = \mathrm{prog}_{\kappa_J} T_J$ (Definition 3.8). The environment δ sends every node variable to its state point; δ' sends every node n to δ_{n^+} with $n^+ := (n_1, \mathrm{nx}(n_2))$ (keeping n_2 when there is no successor).

Value. Mutual recursion on T_J and child lists gives

$$\mathrm{ev}_{\delta'}(P) = \delta_{\mathrm{fold}_{\kappa_J}(T_J^+)},$$

where T_J^+ is the tree with every node replaced by n^+—the same as the proof of Theorem 3.9(1), with base points n^+. By S0.2 the value of $\widetilde{P}$ in the pair environment $(\delta, \delta' - \delta)$ is $(\mathrm{ev}_\delta P,\ \mathrm{eff}_{\delta, \delta' - \delta} P)$, and the update equation (Proposition 3.3) gives $\mathrm{eff}_{\delta, \delta' - \delta}(P) = \mathrm{ev}_{\delta'}(P) - \mathrm{ev}_\delta(P)$. Together the value is $(\delta_F, \delta_{F'} - \delta_F)$ with $F = \mathrm{fold}_{\kappa_J} T_J$ and $F' = \mathrm{fold}_{\kappa_J} T_J^+$.

Components. By induction on T_J: the first component of F depends only on the first components of the nodes and of the children, and is exactly $\mathrm{fold}_{\mathrm{effectFold}}(T) = \tau(T)$; the second component of F is the image tree of T_J under act applied node by node, with root $\mathrm{act}(\mathrm{enc}\, v)$, which decodes back to v by (1).

Charge and payment. By S0.2 and Theorem 3.9(2), $r(\widetilde{P}) = r(P) = \beta(T_J)$, and T_J has the shape of T, so $\beta(T_J) = \beta(T)$. Applying Theorem 6.10 to the reader $o \mapsto (\text{pair environment}, \widetilde{P})$: the paid trace has length $\beta(T)$, every count $k < \beta(T)$ has a strict payment, the target budget is never refilled and the continuation is well founded.

Inverse reading and successor. At every count the expression kept by the installed face is P, and the parser reads back T_J (Theorem 3.9(3)); forgetting the code component reads back T. The recovered root code decodes to v, and its generated next current is the root's generated next current at v. The update morphism $(\mathrm{id}, \mathrm{ev}_{\delta'})$ goes from ev_δ to $\mathrm{ev}_{\delta'}$; on the source side take the relation words of the old trace, and as target fibre element the sum of the relation words of the old and new traces. By Lemma S0.5 the lifting residual vanishes iff the target fibre element is the image of an element of the old fibre; this residual is kept unchanged with the result.

(3) Common history. The source and target histories are histories of the same history law at two occurrences; events, supports, generator closures and relation closures of the common history are the unions of both sides. The two transitions embed the generators of each side by definition. The completion map sends a word u of the generator closure of one side to the class of the same word in the common completion, so its image vanishes iff u lies in the relation closure of the common

history. The evaluators of the two sides are the faithful evaluations of the history law at the two occurrences, reading back the values of the original faithful faces on every generator; relation soundness is equivalent on both sides. The product of the completion carriers Src and Tgt carries the action and observation induced by the two transitions and forms an action model; pairing and dual action come from the scalar pairing of the histories; the measurement $\mathrm{Src} \times \mathrm{Tgt} \to \ell^2(\mathcal{H} \times \mathcal{H})$ together with a linear isometric evolution of that space forms joint integral–coherent action data.

(4) Inverse criterion of the action difference. Write ∂_s, ∂_t for the differentials (measurements) of the two sides; the transition satisfies $\partial_t \circ \iota = \partial_s$.

If there is r with $\partial_s r = \partial_s a_s$ and $\iota(r) = a_t$, then $d = \iota(r) - \iota(a_s) = \iota(r - a_s) \in \operatorname{range} \iota$, so the image residual vanishes, and $\partial_t d = \partial_s(r - a_s) = 0$, so the observation residual vanishes.

Conversely, a vanishing image residual gives r_0 with $\iota(r_0) = a_t$; then $d = \iota(r_0 - a_s)$ and the observation residual $\partial_t d = \partial_s(r_0 - a_s) = 0$, so $r_0 \in \mathrm{Fib}_{\partial_s}(a_s)$.

The computing program is a three-output expression: the difference d, its measurement, and its class in the cokernel; its remaining count is 25. Theorem 6.10 gives a paid trace of 25 steps, each a strict payment, with well-founded continuation, and whole ledger and next current kept at every count.

(5) Literal successor. The target occurrence of step(v) is the event the compiler emits for the next current, equal to the canonical occurrence at the current of v^+, which is the source occurrence of step(v^+). Histories, generator closures, relation closures, exposures and exposure trees are functions of (current, occurrence), hence pairwise equal; target words are carried to next source words along the equality of generator closures, with the underlying word unchanged. ■

Formal correspondence. `Occurrence/Temporal/{Source,Action/Source,Action/Inverse}.lean`; `Occurrence/Temporal/History/Common/{RootSource,Closure,Evaluator/Source,Action/Source}.lean`; `DJ/Joint/{Source,Laws,Consumer,TargetDifference,TargetProgramme,TargetConsumer}.lean`; `DJ/Next/{Material,Pursuit}.lean`. Commit AD.

S3.2 Theorem 9.3: joint recovery by all actors

Objects. The data of Theorem 9.2 and a count. The source and target pairing unfolding trees have all pairing positions as nodes, each with an exposure (M_p, A_p, U_p). Actors are members of the tree traces; the node Hilbert space is indexed by the positions of the traces (the same payload occurring repeatedly is counted at each position).

(1) All-word model. The joint carrier is the product of the carriers of the two sides; the action of a letter ℓ is the carrier action of that actor (source actors act on the first factor, target actors on the second, and "simultaneous" acts by the roots of both sides), and the actions of the root actors are the source and target exposure actions of the original step. The read-out rd is the product of the coarse reader and the measurements of all actors. Lemma S0.6 gives the quotient Mod, the fibre characterization and the completion equivalence; two elements of the completion carrier are equal iff their read-outs after all following words agree, which is S0.6(ii) stated on the completion side.

(2) Ordered effect history. Take $W = \mathcal{H}^{\mathrm{actors}}$, let U_ℓ apply U_p to the component of the selected actor and leave the others unchanged ("simultaneous" applies to the root components of

both sides), and let M be the measurements of the actors. Lemma S0.7 gives that the total residual is the sum of the step residuals transported by the later evolutions, with history length $|w|$; for a single letter the total residual is that actor's covariance effect $U_p M_p(x) - M_p(A_p x)$.

(3) Node Hilbert. The measurement meas : Mod $\to \ell^2(\text{positions}; \mathcal{H})$ takes at every position the feature of that actor (the actor's measurement read through the model). In ℓ^2 the squared norm is the sum of the squared norms of the coordinates, so

$$\|\text{meas}(x)\|^2 = \sum_p \|M_p(x)\|^2.$$

The evolution of a letter on ℓ^2 takes U_p or the identity coordinatewise and is therefore a linear isometry. The node effect unfolds coordinatewise by the definition of the covariance coupling residual.

(4) Future recovery and three faces.

Future recovery. Let $x \in$ Mod be sent to zero by both the coarse reader and all future readers. The vanishing of all future $\mathcal{H}$ readers means: for every prefix bound and every word, every actor's measurement after the word agrees with that at the zero point. The vanishing coarse reader gives agreement of the coarse read-outs after every word. So x and 0 have the same read-outs after every word, and (1) gives $x = 0$. The combined map is injective, Mod is linearly isomorphic to its image, and both composites are identities.

Three faces. With $I =$ Mod (acted on by the model advance), $H =$ the family of ℓ^2 spaces over all prefix bounds (acted on by the evolution at each bound), $Q =$ the coarse model (acted on by the coarse advance), c the future reader and m the coarse restriction, form the complete orbit carrier $\mathcal{C} \subseteq I \times H \times Q$ of Lemma S0.8. The integral and coarse faces are the first and third components. The coherent face sends the second component (the family over all prefix bounds) to the inverse limit along restriction of prefix bounds: restriction at each bound commutes with evolution, so this is the completion map of a compatible family and commutes with the evolution of every letter; each bound coordinate of the completion map is the value of the second component at that bound, so the coherent face vanishes iff the second component vanishes. An element on which all three faces vanish therefore has all three components zero, hence is zero: the three faces are jointly injective, give a double inverse on the image and commute with the action of words (S0.8(ii)).

(5) Paid recovery. For a point, a seed word, a history word and a query word q, the owner-free calculation executes the program generated by the query word, whose remaining count is $|q| + 3$ by the program-budget lemma; Theorem 6.10 gives a paid trace of length $|q| + 3$, and the computed value equals the next value (value-readback lemma). Applying the recovery map to the acted element reads back the next value as first component through the isomorphism of (4). The pair inventory enters the next birth as part of the born inventory, and the same debt is never refilled (Theorem 6.10(2)). ■

Formal correspondence. `AW/{Source,Complete/Source,EffectHistory/Equation,EffectHistory/Source}.lean`; `AW/NodeHilbert/{Source,Future/Recovery/Source}.lean`; `AW/NodeHilbert/Future/Wave/{Words/Kernel,Full/Source,Recovery/Source,Recovery/Consumer,Incidence/Consumer}.lean`. Commit AD.

S3.3 Theorem 9.11: six faces of the same actual material

Common material. Take Paid material M of the original actual frame. It retains occurrence o, environment ρ, original Raw e, paid state e', full Trace $T : e \to e'$, owner and original increment ε. The relation word $\eta = T.\text{relationWords}$ is generated step by step by that Trace. Each step's relation image is $\delta_{e_h} - \delta_{e_{h+1}}$, and their sum is

$$\operatorname{rel}_\rho \eta = \sum_{h<|T|} (\delta_{e_h} - \delta_{e_{h+1}}) = \delta_e - \delta_{e'} =: b.$$

Let H be this actual paid prefix's prefixHistory. Every relation witness comes from that prefix, so the sum belongs to H's relation closure. The actual relation $\sigma = (b,\, b \in H.\text{relationClosure})$ is therefore a concrete element of this history. Projecting b to actual history completion gives zero.

Six faces and the complex. The algebraic face retains $(e, e', e \ominus e')$ and the complete coefficient pairing; the combinatorial face retains T, every step's exposure and its position; the topological face is this actual history and its closure; the logical face is the following chain condition. Let the first differential d_1 be relation-closure inclusion and the second d_0 history completion projection:

$$C_1 \xrightarrow{d_1} C_0 \xrightarrow{d_0} \widehat{C_0}, \qquad d_1(\sigma) = b, \qquad d_0 d_1(\sigma) = 0.$$

The same holds on the actual history's relation closure, giving zero composite. The relation face stores this actual σ; the cochain face stores this actual complex with the same boundary. All six faces are constructed from one M, so their occurrence and Raw readings agree by definition.

Effect and whole-word recovery. S0.3 gives

$$\operatorname{ev}_{\rho+\varepsilon} b = \operatorname{eff}_{\rho,\varepsilon} b = \operatorname{ev}_{\rho+\varepsilon}(e \ominus e').$$

In the quotient by updated evaluation's kernel, the canonical class is zero exactly when the right side is zero. The standard class-to-range isomorphism reads it back as original native value, giving the exact inverse/residual criterion. Whole-word recovery uses the original complete word reading, whose kernel is zero: if all basis-coefficient readings vanish, every coefficient vanishes. Coimage recovery thus satisfies $\operatorname{rec}(\operatorname{can}(w)) = w$ for every w, while the combinatorial face still retains full Trace.

Across literal Born. Original admission's inventory callbacks embed this paid material into actual Born using preserve; the stock outcome's Trace is this original T. The original settlement compiler supplies full next root/current identity. Transport Raw's projection, active, classifier and payload equalities along that identity to obtain the next actual installed query. Source material transports its raw/environment/state/Trace/owner together. If actual Born environment is ρ_B, the registered request still uses the original registered environment ρ, with increment $\rho_B - \rho$. This transport changes only the increment: original σ, b, T remain retained and updated effect is $\operatorname{eff}_{\rho,\rho_B-\rho} b$. Normalizing the new request at ρ_B generates new σ_B and a new boundary. Original inventory retains both origins without asserting $\sigma = \sigma_B$. ■

Fixed source and consumer. MV3's `ACT/Receipt/Configured/Writeback/Successor/FourFace/{Source,Consumer,Native/Source,Native/Consumer}.lean`; `Operations/Inquiry/Cofinal/Nativ`

`e/{Source,Consumer,Registered/Source,Next/Source,Next/Consumer,Installed/Source,Installed/Consumer,Cofinal/Source,Cofinal/Consumer}.lean`. Whole-word action and the all-future model in S5.4 consume the original next packet's installed material.

S4 Theorems 9.5–9.7

S4.1 Theorem 9.5: mother admission

Objects. An inquiry frame Φ and a source program $\mathcal{S} = (L, \text{reader}, \ldots)$ (Definition 9.4). The shared runtime installs Φ and the reader of $\mathcal{S}$ as an inquiry state and answers its unique query; s is this answered state, o its current occurrence, and ρ' the environment of the raw input of the query of the shared runtime's literal next state.

(1) Material and value. The material is assembled by definition: environment ρ and expression e are the output of the reader at o, the increment is $\varepsilon = \rho' - \rho$, the paid state (e', t) is the result of the shared runtime at o, and the bearer is the bearer of this query at s. The registered input is the residual request of Definition 8.11, with environment $\rho + (\rho' - \rho) = \rho'$. Value and residual are given by S2.1(1).

(2) Necessary admission. If the initial state of the residual request were a settlement, its budget would be zero (the execution debt law has budget zero at settlements); but the residual request has budget $r(e) + r(e') + 2 > 0$ (S2.1(2)), a contradiction. So the source's first action is an execution step. The birth program is generated by the old state, the query, the registered input, the source packet, the authority and this step; on the root and visit of s, the compilation program sends the unique query to what the birth program generates on the exact temporal causal event—an obligation admission—and the equation holds by definition.

(3) Target. The admission target carries: a literal next current equal to (vocabulary, root, visit) of the target state; the first whole-ledger write; and, for every old projection, an outcome on the target equal (heterogeneously) to the original outcome. Validity of the successor is given by the general criterion "an active node preserves the generated living law under an admission successor", whose hypotheses are exactly the literal next current and the compilation equation above.

(4) Born frame. The born frame takes s as old state, the residual request as registered input, the reader environment, depth zero, and as inventory the complete exposure of the paid trace. Its current state is by definition the admission target state. The runtime of Theorem 8.12 starts from it: the erasure of the initial node equals the erasure of the target presentation; the macro successor and preservation of the living law at every tick are the node equations of S2.1(3); no refill and well-foundedness are S2.1(4) and Theorem 6.10(2).

(5) Next content. By (S0.1) and the update equation, the sum of (old, effect) of the paid result equals the program's value in the updated environment:

$$\mathrm{ev}_{\rho}(e) + \mathrm{eff}_{\rho,\varepsilon}(e) = \mathrm{ev}_{\rho+\varepsilon}(e).$$

The environment of the born frame at every occurrence is given by the environment read-out of this value. The macro-history program satisfies $\mathrm{next}(\mathrm{hist}_k) = \mathrm{hist}_{k+1}$; the active environment of

the initial frame is already the next value of the history of tick k, i.e., tick $k+1$, and the born frame advances one more tick, giving tick $k+2$. The born inventory is the side-by-side seed of the old inventory and the paid exposure, both of which are parts of it (left and right embeddings).

(6) Native shift. The native cursor shift of the query frame of the same stage gives a new frame; applying (1)–(4) to it gives compilation, first whole-ledger write, validity of the successor, literal successor, the run at every offset, no refill and well-foundedness; the shifted environment is the actual environment of the cursor shift, and effect and inverse reading are read from the residual request. ■

Formal correspondence. `ACT/Programme.lean`, `ACT/Receipt/{Source,Execution,Compiler,Consumer,Runtime}.lean`, `ACT/Target.lean`, `ACT/Receipt/Faces/{Source,Action,Consumer,Native/Source,Native/Consumer}.lean`; next content `AW/NodeHilbert/Future/Wave/Incidence/Environment/{Source,Consumer}.lean`. Commit AD.

S4.2 Theorem 9.6: typed inventory crosses the admission

Objects. The setting of Theorem 9.5; the physical raw input (ρ_P, e_P) (variables of physical type Var_P) with its paid result, and the low-level raw input (ρ_L, e_L) (variables in L) with its result. The binding is $\sigma : \mathrm{Var}_P \to \mathrm{Expr}_L$, $\sigma(x) = \mathrm{const}(\rho_P(x))$.

(1) Binding preserves values. Structural induction proves $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$: at a variable $\mathrm{ev}_{\rho_L}(\mathrm{const}\,\rho_P(x)) = \rho_P(x)$; at a constant both sides agree; addition, linear and bilinear nodes agree term by term by the induction hypothesis. Formal words extend linearly. Hence the value of the low-level query $(\rho_L, e_P[\sigma])$ equals that of the physical query. The substituted trace is the physical paid trace substituted along σ, starting at $e_P[\sigma]$, with its relation boundary substituted accordingly; the physical paid state is a constant and remains a constant after substitution. Every execution trace ending at a constant has length equal to the remaining count of its start (Proposition 3.7(1)), so the substituted trace has length $r(e_P[\sigma])$.

(2) Registration and readback of the inventory. The complete material (both raw inputs and results, the binding, both written inventories and the substituted trace) is installed as one projection on the base root of the frame, with payload type the type of the material. The projection law of the mother admission target inherits the old projection law (Theorem 9.5(3)), and its outcome at this projection is heterogeneously equal to the original outcome; the equality of types is given by the same heterogeneous equality, and transporting the outcome along it to the material type gives the material itself.

(3) Born inventory. Every event of the physical inventory is carried into the low-level language by substitution along σ and placed beside the low-level inventory to form the born inventory; the left and right embeddings ensure that every event of both sides is in it. The born frame runs as in Theorem 9.5; the next inventory extends the born inventory, and the coordinate map is injective under both actions mathNext and nextBorn and reads back every original event.

(4) Inverse readback. Index a word family W by the events of the born inventory, with joint map $W \mapsto (\mathrm{index} \mapsto (\mathrm{ev}_\rho, \mathrm{eff}_{\rho,\varepsilon})(W_{\mathrm{index}}))$; the canonical residual of the actual word family vanishes iff its image vanishes (S0.4), and that image is the result value. The next inverse readback is the map induced by the update morphism (S0.4). At every tick of the cofinal stage, the pair value of

a relation word in that tick's environment reads the value of the next tick's field at offset zero; the paid trace has length equal to the remaining count of the raw input. ■

Formal correspondence. `ACT/Receipt/Inventory/{Source,Laws,Installation}.lean`; `Inventory/Born/{Source,Consumer}.lean`; `Inventory/Born/Vector/Fibre/{Source,Consumer}.lean`; `Inventory/Born/Coordinates/Actual/{Source,Query}.lean`; `Coordinates/Actual/Cofinal/{Source,Consumer}.lean`. Commit AE.

S4.3 Theorem 9.7: word–state pairing and paid prefixes

Objects. A runtime and its source; the old environment ρ_s and increment $\varepsilon_s = \rho_{\sigma(s)} - \rho_s$ of a state s. The binding σ of orbit variables satisfies two readbacks: $\mathrm{ev}_{\rho_s}(\sigma x) = \rho_{\sigma(s)}(x)$ and $\mathrm{eff}_{\rho_s,\varepsilon_s}(\sigma x) = \varepsilon_{\sigma(s)}(x)$.

(1) Pairing. On base points δ_s, $\langle w, \cdot\rangle$ takes the stage inventory $(\mathrm{ev}_{\rho_s} w, \mathrm{eff}_{\rho_s,\varepsilon_s} w)$, extended uniquely and linearly by the universal property of free modules (Proposition 2.3); linearity in w comes from linearity of evaluation and effect.

(2) Covariance and coimage action. On base points,

$$\langle w[\sigma], \delta_s\rangle = (\mathrm{ev}_{\rho_s}(w[\sigma]),\ \mathrm{eff}_{\rho_s,\varepsilon_s}(w[\sigma])) = (\mathrm{ev}_{\rho_{\sigma(s)}} w,\ \mathrm{eff}_{\rho_{\sigma(s)},\varepsilon_{\sigma(s)}} w) = \langle w, A\delta_s\rangle,$$

where the middle step is the substitution lemma—evaluation after substitution equals evaluation in the environment "variables take the values of their substitutes", and likewise for effects—and the two readbacks identify that environment with $\rho_{\sigma(s)}$ and $\varepsilon_{\sigma(s)}$. Both sides are linear on $\mathbb{Z}[S]$, so $\langle \sigma^*(\cdot), \cdot\rangle = \langle \cdot, A(\cdot)\rangle$ on the whole carrier. This says (σ^*, A^*) is a morphism from the pairing to itself, which induces the action on the coimage by S0.4; the read-out formula on $\mathrm{can}[e]$ holds by definition.

(3) Iteration is time. Induction on n: $n = 0$ is trivial; the step uses the base-point equation of (2) at tick d and the induction hypothesis at $d+1$. $\mathrm{act}^n[e] = [e[\sigma]^n]$ follows from commutation of the induced map with the canonical map.

(4) Prefix charges. The i-th prefix executes $\widetilde{e[\sigma]^i}$, and by S0.1 and S0.2 its trace has length $r(e[\sigma]^i)$; summing gives Σ_n. The batch query places the $n+1$ terms in a vector-valued language: each term enters through one coordinate-embedding node and one accumulation node, so the remaining count is the sum of the terms plus $2(n+1)$; execution length equals remaining count (Proposition 3.7(2)). Theorem 6.10 gives strict payment at every step and same-debt lineage.

(5) Call recovery. The actual query material of a call frame at count $c + i$ gives the environment there; the old values and increments of the snapshots needed are taken entry by entry from these environments (an increment is the difference of two adjacent ones). The batch environment so assembled equals the orbit batch environment coordinate by coordinate by definition, so the result values of the two queries are equal; the paid trace of every call has length equal to its remaining count (Theorem 6.10(3)), and summing gives call charge equal to call cost. ■

Formal correspondence. `Operations/Inquiry/Context/Native/Pairing/{Source,Orbit/Source,Orbit/Action,Orbit/Iteration,Orbit/Iteration/Batch,Orbit/Iteration/Current}.lean`;

calls and history ⋯`/Batch/Calls/{Source,Consumer,History/Source,History/Consumer}.lean` (full prefix in Appendices D.2–D.3 of the main text). Commit AE.

S4.4 Theorem 9.9: complete recovery, both inventories and Trace positions

The original model first recovers the entire carrier. Let S be the original sealed run's state set and $\mathcal{F} = \mathbb{Z}[S]$. Its original complete action-word model has source map q. Its completeRead includes the original whole-word reading at the empty action word, so $\mathrm{rec}(q(w)) = w$ for every $w \in \mathcal{F}$. Define O on basis states by reading their actual registered frame and extend linearly: $O(\delta_s) = \delta_{\mathrm{readFrame}(s)}$. The sealed runtime state at tick i stores the original recursive index; node equality and erasure injection give $\mathrm{readFrame}(s_i) = \Phi_i$. The original full model's point-action identity $Aq(\delta_{s_i}) = q(\delta_{s_{i+1}})$ therefore gives

$$O\mathrm{rec}(q(\delta_{s_i})) = \delta_{\Phi_i}, \qquad O\mathrm{rec}(Aq(\delta_{s_i})) = \delta_{\Phi_{i+1}}.$$

Applying mapDomain to $(\mathrm{inventory}, \mathrm{pairInventory})$ gives the point word of the exact inventory pair. If model points i, j agree, recover their original states, then read nodes and erasures to obtain $i = j$. This injection follows from original strict frame rank; the earlier paid state survives before Frame restriction.

Complete stock. The original query's raw input, value, Trace, effect, inverse, decoder, whole and literal next come from the same installed reader at the actual occurrence. With absent calculationReader it is registered Raw; with a calculationReader it is that actual reader. These two cases are distinguished. Original stock programme passes this material to mother admission and preserves both original inventory callbacks. The Type completion receipt selects the actual settled Born, and the original sealed node reads its complete frame at distance plus one.

Positionwise embedding. Let H_Q be this query's source-written exposure, H_o old written, H_m migrated written and H_P physical written. Original complete written and Born inventory are

$$H_C = \mathrm{seed}(\mathrm{seed}(H_o, H_Q), H_m), \qquad H_B = \mathrm{seed}(H_P, H_C).$$

The full-tree equality between original source exposure and actual query exposure first transports $j \in \mathrm{Fin}(|H_Q|)$ to sourceWritten. Three common inclusions then place H_Q in the inner seed's right side, the inner seed in H_C's left side, and H_C in H_B's right side. Compose the original left/right seed readbacks to obtain

$$H_B.\mathrm{trace}[\iota(j)] = H_Q.\mathrm{trace}[j], \qquad \iota(j) = j + |H_o| + |H_P| + 3.$$

Each seed seam occupies one position, giving offset three. Equality $\iota(i) = \iota(j)$ cancels the common natural offset and gives $i = j$; $i < j$ gives $\iota(i) < \iota(j)$ by adding that same offset. The map is thus injective and strictly order-preserving, retaining equal-valued events at distinct positions.

The two reading locations of original next. Actual settlement's debtAdmission recovers complete material at the admission's initial visit and gives the full next packet. After-initial read at the old initial visit retains original installed material; after-current read at the new current is the next query sent by original compilation. Each transports along its own full-current identity

and returns its respective original reader. Shared source material does not identify these different visits as one query. ■

Fixed source. MV2's `Operations/Inquiry/Context/Native/Frame/{Source,Stock,Stock/Return,Stock/Coordinates,Stock/Birth}.lean`, with original fullword, model-next, inventory-next, source-exposure, intoBorn, after-initial and after-current declarations and direct consumers read at MV3.

S5 Actual total value, complete continuation and original whole-word action

S5.1 Theorem 9.8: the default run executes the source-generated request and its subtrees

(1) Source-generated request. Take the root and installed reader of the joint program of Theorem 9.2, form the source inquiry frame of the owner-free installation, and take as seed the zero unfolding of the generator of the registered expression. The source program of the default runtime has "the original request of this occurrence" as reader: at an occurrence o the reader returns the request expression and environment generated for o. Apply Theorem 9.5 to this source program: the raw input of the query is the request read by the installed face (by definition); the result value equals the execution value of the request (the completed value of Theorem 6.10 together with soundness of ev); the paid trace has length equal to the remaining count of the registered input (Theorem 6.10(3)); by Theorem 9.5(5) the born pair inventory consists of the previously written part and the part paid now; the actual query, answer and successor at every count are node equations of the runtime. The ledger roots agree by construction. In the branch version the core and relation expressions are the two coordinates of a binary batch, and the trace and substituted trace of the batch execution enter the initial inventory.

(2) Subtrees enter the main expression. The main expression is a sum of three terms: the original query expression, the subtree at the old code and the subtree at the next code, each entering the output type through a coordinate embedding. The subtree at a code is the zero constant if recognition has no successor at the visit of that code, and otherwise the embedding of the raw input of the subprogram at that code. Hence

$$\mathrm{ev}(\mathrm{main}) = (\mathrm{ev}(\mathrm{query}),\ \mathrm{ch}(c_{\mathrm{old}}) + \mathrm{ch}(c_{\mathrm{next}})),$$

where the first component of a subtree value is zero. Remaining counts add term by term, plus two additions and one linear embedding, three combining nodes in all: $r(\mathrm{main}) = r(\mathrm{query}) + c_{\mathrm{old}} + c_{\mathrm{next}} + 3$. Paid trace length: the trace of the owner-free installation has length equal to the remaining count, and the query's own paid trace has length $r(\mathrm{query})$; substituting gives the claim. The executions of the two subtrees have length equal to their own remaining counts (Proposition 3.7(2)). Events of the old inventory and of the new payment are kept by the embeddings of the side-by-side seed; the sum of the old and next actor components is the state point of this tick's source operation, read back through the complete actor carrier.

(3) Subtree environments. At a supplied occurrence the subtree environment is defined by the read-out of the original binding in that environment; the environment of the born frame is the actual updated environment (Theorem 9.5(5)), and the subtree environments are read from it by

the same binding. Scalar and pair inventories are kept as in (2); the initial node and the input at every tick of the runtime are the runtime's defining equations. ■

Formal correspondence. `ACT/Inventory/Pair/Residual/Joint/Past/NextObservation/Request/Acted/{Source,Consumer}.lean`; `DJ/{Joint,Branch}/Primary/{Source,Consumer}.lean`; `DJ/Branch/JointQuery/Actor/{Source,Embedding,Action/Source,Operation/Source,Operation/Consumer}.lean` and `Actor/Operation/Query/Inventory/{Language,Source,Consumer,Action/Source,Action/Consumer,Live/Source,Live/Consumer}.lean`; default successor `DJ/Next/{Source,Consumer}.lean`. Commit AE.

S5.2 Theorem 8.15: actual total value, a decoder per receipt and Born

Same-occurrence actual material. At the exact occurrence o, the original action reader supplies Raw (ρ, e). Its original result supplies paid state $\mathrm{const}(p)$, complete Trace and actual $p = \mathrm{ev}_\rho e$. Original material M retains these objects and increment ε, the difference from actual afterEnvironment. New request $N(M)$ is $e \ominus \mathrm{const}(p)$, registered at $\rho + \varepsilon$. Hence

$$\mathrm{ev}_{\rho+\varepsilon} N(M) = \mathrm{ev}_{\rho+\varepsilon} e - p =: n, \qquad p + n = \mathrm{ev}_{\rho+\varepsilon} e.$$

The first identity expands subtraction constructor by constructor; the second is group cancellation. p and n are respectively the old complete query's paid value and the original residual's actual completed value; the updated target value is generated.

The actual programme change. Preserve original lowProgramme's source, reader, calculationReader, effect/inverse, whole and both inventory callbacks. Replace only `nextEnvironmentReadAt` with $\mathrm{write}(p + \nu(o'))$. Eliminate the actual native event to define ν: zero on a paid branch, its settlement value on a settled branch. The new programme follows the original frames. At actual settlement event o_B selected by the original Type receipt, $\nu(o_B)$ is exactly n. Expanding the nextBorn environment field of original settled compilation gives

$$\mathrm{env}(\mathrm{nextBorn}) = \mathrm{write}(p + \nu(o_B)) = \mathrm{write}(p + n).$$

A paid branch's original compilation target is mathNext. It does not invoke this birth environment; it preserves original registered input, state and Trace while taking another step. Thus the decoder field belongs to the programme and original settlement performs the actual writeback. The total-value identity uses only ρ, e supplied by the same actionReader and its paid endpoint. For the actual Contextual/Incoming source family of S2.3, the separately generated environment square also aligns original queryReader environment with this actualMaterial.

Regeneration per packet. Let P_j be packet j of the original Factory. Retain both receiver inventories, original calculation reader and original language. The receipt above supplies actual Born B_j; next packet P_{j+1} forms its new seed from this Born's complete material and registered expression. `nextCfg` reads original actionResult again at this actual next packet, obtaining p_{j+1}; it reads ν_{j+1} on its actual residual events and installs $\mathrm{write}_{j+1}(p_{j+1} + \nu_{j+1})$. Applying the preceding argument to P_{j+1} gives

$$\rho_{B_{j+1}} = \mathrm{write}_{j+1}(p_{j+1} + n_{j+1}).$$

This uses the new p_{j+1}. Two-generation consumers transport original language along the full next-packet equality, recover B_j, and obtain the same construction for `secondNextCfg`. Induction on j supplies a fresh decoder at every receipt and preserves both inventories. ■

Zero-effect control. The original configured query const$(1,0)$ has $p=(1,0)$ and $n=0$; actual Born writes $(1,0)$. The next paired generation's original paid value is $((1,0),(0,0))$, again nonzero, and the regenerated decoder preserves it. This actual control distinguishes native effect from total value.

Fixed source. MV3's `ACT/Receipt/Configured/Writeback/{Source,Consumer}.lean` and `Writeback/Successor/{Source,Consumer}.lean`, including `old_paid_value`, `native_value`, `updated_total`, `born_environment`, `successorPacket`, `second_decoder` and `second_writeback`; original comb consumers `Feedback/ClozelOriginalKConfiguredPairWriteback.lean`, `Feedback/ClozelOriginalKSourceSuccessor.lean` and their Verification consumers.

S5.3 Theorem 9.10: complete cofinal segments of the same run

Two stages of the original run. Actual Stage is a dependent sum: admission retains grade, complete datum, configuration and language; continuation retains these objects and the original paid Frame. Original advance sends an admission packet's actual admission into continuation. A paid continuation branch takes only mathNext, preserving grade and packet material. A settled branch takes original nextBorn, adds its registered expression to the next seed and returns to admission. Inventory preserve retains old scalar inventory, old pair inventory and the actual generated inventory at each level. Actual actionReader still uses original calculationReader when present, otherwise original reader. The entire Stage is one source-generated function.

Type data for the next packet. From any tick t, eliminate its actual Stage. An admission returns distance zero, original packet and frame equality. A continuation invokes the receipt of S1.3 in this same recursive frame run. It supplies exact distance B, all B payments and the original settled frame. Original advance reaches admission at the following tick, giving distance $B+1$. Store every intermediate paid Frame and original Stage equality in the segment. The strict rank is (inventory size, causal depth plus one): admission/settlement grows inventory; payment preserves inventory and increases depth. Thus source erasure is injective on original ticks.

Stop construction and cover. Let $d(t)$ be this actual distance, $t_0 = d(0)$ and $t_{j+1} = t_j + 1 + d(t_j+1)$. Its distance data supply original Stage admission equalities at both endpoints. Each historyAt is the original frame at tick t_j+h, $0 \le h \le \ell_j$, where $\ell_j = 1+d(t_j+1)$. Consequently

$$t_j + \ell_j = t_{j+1}, \qquad t_{j+1} \ge t_j + 1, \qquad t_j \ge j.$$

For any t, taking $j=t$ gives $t \le t_j$, proving strict cofinality. For segment cover choose the first stop beyond t; historyAt from the preceding stop contains t. Source-data equality of adjacent packets is assembled fieldwise from actual admission, nativeAfterAdmission and bornAfterNative. Grade advances at packet admission transition and remains distinct from actual tick.

Advance on the complete original word. Let W_t be the original runtime's complete field word. Original point action gives $A_{\mathcal{R}} W_t = W_{t+1}$. Induction on h gives $A_{\mathcal{R}}^h W_t = W_{t+h}$; at $h=\ell_j$, the index equality gives Theorem 9.10(3). Free-carrier linear extension makes original complete

source word, field word, cofinal-point injection and word–state dual commute along this same source span. The segment's paid trace and admission stock remain in this carrier. ■

Fixed source. MV3's `F/Admission.lean` and `F/SourceFamily/{Runtime,Activation/Source,Activation/Stage,Activation/Admissions}.lean`; Appendix D.3 expands `F/`. Complete-word consumers are `Operations/Inquiry/Cofinal/{Source,Consumer}.lean`, with source-span, strictness, cofinality, cover and field/point commutation declarations.

S5.4 Theorem 9.12: complete original word action and the all-future model

Original language at each real packet. Installed Raw, query incidence and full packet equality at stop j come from S3.3 and S5.3. Its complete word module $\mathcal{W}_j$ has native reading

$$d_j(u) = (\mathrm{ev}_{\rho_j} u, \mathrm{eff}_{\rho_j,\varepsilon_j} u).$$

Pair-lift each expression node by the original lift, then transport along the original next-packet language equality. This defines original `sourceWordAction` S_j. Appendix A.2 gives paired evaluation on every expression; linear extension gives

$$\mathrm{ev}_{\rho_{j+1}^{\mathrm{receiver}}}(S_j u) = d_j(u).$$

Receiver old/increment coordinates are read from its original registered environment; full next-packet equality identifies grade, seed, Frame and language. This is the actual receiver square and does not infer $\rho_{j+1} = \rho_j$. Updated environment $\rho_j + \varepsilon_j$ still uses the original material's actual increment; old reading plus effect generates the completion update at this stage.

Inventory and finite support. A member of original scalar relation stock enters born relation stock by pair lift. Original pair stock and every paid prefix remain unchanged under mathNext and enter nextBorn through preserve. Original seed inclusions supply inventory membership under source action. A word's inventory membership means actual relation-stock membership or generation by actual stock; the assertion keeps this premise. Let $\mathcal{W} = \bigoplus_j \mathcal{W}_j$. For finitely supported w, define

$$S(w) = \sum_{j \in \mathrm{supp}(w)} \mathrm{in}_{j+1}(S_j w_j), \qquad D_j(w) = d_j(w_j).$$

The support remains finite under $j \mapsto j+1$, so S is well-defined and linear. Original packetwise transports identify it with original common action. All site maps form jointObserver. Evaluators on finite prefixes $0, \dots, b$ commute with deletion of the last site and generate original joint completion. Its fibre is exactly characterized by each site's lifted difference belonging to that site's original pair-relation range.

All future words and the maximal kernel. The allowed alphabet is $\{*\}$, with this original S as its action. Put $S_{[]} = \mathrm{id}$ and $S_{a*} = S \circ S_a$, and define

$$K_{\mathrm{fut}} = \bigcap_{a \in \{*\}^{<\omega}} \bigcap_j \ker(D_j \circ S_a).$$

It is a submodule and $S(K_{\mathrm{fut}}) \subseteq K_{\mathrm{fut}}$. For $w \in K_{\mathrm{fut}}$ and any a, j, $D_j(S_a S w)$ reads a future word with one additional original letter, so vanishes. The quotient map q_{fut} therefore admits unique model action $\bar{S}$ and descended readings with

$$\bar{S} q_{\mathrm{fut}} = q_{\mathrm{fut}} S, \qquad \bar{D}_{j,a} q_{\mathrm{fut}} = D_j S_a.$$

Two quotient classes agree exactly when $u - v \in K_{\mathrm{fut}}$. Expanding the kernels gives equal readings for every allowed future word at every site. If L is invisible now and $S(L) \subseteq L$, induction on word length gives $S_a L \subseteq L$; present invisibility then gives $D_j S_a L = 0$, hence $L \subseteq K_{\mathrm{fut}}$. This proves maximality using all allowed future words.

Returning to original completion. The empty word gives $K_{\mathrm{fut}} \subseteq \bigcap_j \ker D_j$, so sending $q_{\mathrm{fut}}(w)$ to original jointCompletion's source map $q_{\mathrm{now}}(w)$ is well-defined. Composing with $\bar{S}$ returns precisely $q_{\mathrm{now}}(Sw)$. The actual source span of S5.3 connects cofinal-packet word action to complete State action on all original ticks. All-future Model, original completion update, receiver square, actual inventories and literal next lie in this same original source chain. ■

Fixed source. MV3's `Operations/Inquiry/Cofinal/Native/Installed/Cofinal/Tower/{Source,Consumer,Action/Source,Action/Consumer,Action/Common/Source,Action/Common/Consumer}.lean`, with original maximal-kernel proofs in `Operations/Observed/Words/{Kernel,Model}.lean`, all under `GenericFoundation/`.

S5.5 §10.1: actual comb-value feedback and the complete R

Input and installation of the original calculation. Original observation supplies nontrivial zero coordinate ζ, its exact Xi order m and original comb q_0. The original operator's resolvent parameter is $z = \zeta/2$; the original right-half proof $\Re\zeta > 1/2$ gives $\Re z > 1/4$. Define $e_0 = x$ and $e_{k+1} = \mathrm{linear}_R(e_k)$. Induction on k gives

$$\mathrm{ev}_\rho e_k = R^k(\rho(x)), \qquad r(e_k) = k + 1.$$

Execution at $\rho(x) = q_0$ produces actual $v = R^m q_0$ and its complete Trace of length $m + 1$. Before emitting, the original named facade installs this result law at the same occurrence of its original authority source. Its calculation face reads z, m, q_0, e_m and actual result from that occurrence. The remainder of the original projection extension preserves the complete old ledger/emitter, retaining physical next and lower authority. Physicality of original v follows from the original Xi-order consumer.

Actual value generates the increment. Take that actual v and set $\rho^+ = \mathrm{constEnv}(v)$, $\delta = \rho^+ - \rho$. Paid material retains original e_m and $\mathrm{const}(v)$, giving request value

$$\mathrm{ev}_{\rho+\delta}(e_m \ominus \mathrm{const}(v)) = R^m v - v.$$

Its expression has positive budget. Original lowProgramme's pair lift writes the full updated value through decoder $(a, b) \mapsto a+b$. The Type receipt of S1.3 and S5.2 give actual nextBorn environment $\mathrm{constEnv}(R^m v)$. Here δ denotes the comb increment generated by the calculation result; generic

actualMaterial's ε is actual afterEnvironment minus original environment. They agree once this configuration's environment equality is proved.

Three primitive steps of full R. Let R_z be the original ambient resolvent and P the original closed-subspace orthogonal projection. Paired input is $(v, 0)$. The original programme performs R_z and then splits: variable read $\to (v, 0)$, linear $R_z \to (u, 0)$, split $\to (p, r)$, where $u = R_z v$, $p = Pu$ and $r = (1 - P)u$. Each of the three nodes pays one step; complete Trace retains its second node $(u, 0)$. Equality $p + r = u$ is the original complementary-projection identity. Physical decoder takes p and complete stock retains r and the entire programme.

Original Mellin pricing. In the original parameter range $\Re z > 1/4$, $1/4 < \Re w < 1/2$, let L_w be the original completed ambient Mellin functional. Continuous linear L_w commutes with the original resolvent's Bochner integral. The original dilation law reads its integrand as weight times character times $L_w(v)$. Their product kernel is $\exp((1/2 - z - w)h)$ and has actual decay $\Re(1/2 - z - w) < 0$. Thus

$$L_w(R_z v) = -\int_0^\infty e^{(1/2-z-w)h}\, dh\; L_w(v) = -\frac{L_w(v)}{z + w - 1/2}.$$

In this range the original Gamma–Mellin bridge cancels normalization using $\Gamma_{\mathbb{R}}(2w) \neq 0$, giving physical heat $H_w(v) = 2L_w(v)$ and $M_w(r) = L_w(r)$. Apply the linear reading to $p + r = R_z v$ and multiply by two:

$$H_w(p) + 2M_w(r) = -\frac{H_w(v)}{z + w - 1/2}.$$

Pricing therefore consumes complete $R_z v$ and its original split, including the full remainder.

The original next query. Actual configuration's calculationReader installs this same three-step programme. Frame/stock recovers actual Born and next Raw. The next paired input comes from literal physical Born's p; its actual result is $(PR_z p, (1-P)R_z p)$, still with Trace length three. In the same registered environment this result's residual value is zero. S5.2 nevertheless writes back complete oldPaid. The next successor packet regenerates its decoder from its own actual result. Original programme, complete Trace, both inventories and whole-word Model remain retained together. ■

Fixed source. The calculation face originates at MV2; actual feedback and full-R consumers at MV3. Under `NoIslandNoMagic/CanonicalRiemann/GeneralizedDual/CoPoissonEnergy/Muntz/Cokernel/CenteredGram/JointAction/Runtime/`: `Material/Calculation/ClozelOriginalKCombSourceCalculation.lean`, `Root/ClozelPairedOmegaEffectRowRoot.lean`, `Facade/ClozelPairedOmegaEffectRuntimeFacade.lean`, `Feedback/ClozelOriginalK{ReceiptSource,ReceiptFeedback,PhysicalPricing,PhysicalAction,PhysicalFeedback,RegisteredSourceFeed,ConfiguredPairWriteback,SourceSuccessor}.lean` and original Verification consumers.

S5.6 §10.1: 37 primitives to the body at every tick and finite paid history

Original input and Type 0 installation. Fix original primitive input I with 37 entries. Original finite programme executes translation, repair, resource response and chemical exchange in sequence. `generated_input(I)` returns this computation's actual occurrence. Original positive completion excludes failure branches; the successful body is generated by this run. The dependent type

$\mathsf{InputBodyAt}(I)$ eliminates the original result: absent programme stores exact missing disposition; retained failure stores that original failure; successful response satisfying original profile stores the body from that actual disposition; failed profile stores its original residual. The fixed source instance's original positive proof recognizes its actual successful branch. DNA, full coding, protein, nuclear/electronic fields, resource account, CP focus and open responsibility, full inventory and written history are consequently generated from original value fields.

Original Native current is (phase, I, B) with $B : \mathsf{InputBodyAt}(I)$. Original update computes actual body event and bodyWrite on this body. Its target is

$$\text{next}(\text{phase}, I, B) = (\text{phase} + 1, I, \text{bodyNext}_I(B)).$$

Original Write stores actual body and the equation identifying it with next-body; original Root compilation reads this Write. Literal next's body is therefore this tick's actual written body. Inducting on tick, the input stays I and the running body is repeated original bodyNext applied to original B. Its higher source evidence transports as a body type index. Original Delivery's readNativeBody/readNativeWrite/readWrittenNativeBody equal this Root.current, emitted Write and target field by field.

Full smoothness and time refinement. At actual PostState, electronic occupied matrix and nuclear pose use original half-step Cayley update. Complete raw nuclear/electronic energy includes all original nuclear terms, electronic trace and two-body quartic terms. The four stored/local cases of every Gaussian nuclear integral admit differentiation under the integral; finite sums over nuclei and matrix entries preserve differentiation. Every nuclear-pair distance is nonzero at separated pose. Thus original fields generate full smoothness at the actual state. Let $\pi(t)$ be price of the original ready candidate. Half-step coordinates tend to original occupied, full force is continuous and nuclear displacement tends to zero, so sufficiently small time preserves separation. Complete endpoint occupied and full raw energy are continuous, yielding

$$\pi(0) = 0, \qquad \lim_{t \to 0} \pi(t) = 0.$$

Original refinement preserves actual fields, stock and account while replacing timestep by $t_k = t_0\, 2^{-k}$. Readiness holds eventually.

A source-generated finite disposition. If reserve $A > 0$, continuity gives a k with readiness and $\pi(t_k) < A$, so the weak guard ready and $\pi(t_k) \le A$ has a witness. Original construction uses Nat.find to take the first satisfying k_*. It returns a Type disposition retaining k_*, actual event, complete afterState, updated inventory/account and result equality. Every $k < k_*$ is unpaid. The original body event on that branch equals original refineState; induction identifies the state of k actual bodyNext steps with this refinement state.

If $A = 0$, first take the first ready k_*, whose finite existence still follows from small-time separation. The actual guard then tests $\pi(t_{k_*}) \le 0$. If true, it generates paid; otherwise it generates exact energyShortage, retaining the first-ready and positive-price witnesses. Each earlier $k < k_*$ is not ready and is again an unpaid refinement. Both branches return finite source-generated Type dispositions; zero reserve is not assumed to permit payment.

Fulfilment at every original Root tick. Induct on steps k from any actual body. Original bodySteps' complete body is heterogeneously equal to the body reached by k literal next steps of the same sealed runtime; preserved input I provides the original index for transport. At k_*, the selected actual bodyEvent is original Delivery's NativeWrite at actual tick k_*. Written body is original bodyNext, and the following original tick reads that same body. The segment includes original compilation and body fields of every preceding unpaid event. The paid branch preserves original charge/electron count, particles and fields, and complete $E+A$ conservation while advancing actual elapsed; unpaid refinement preserves elapsed by original law. Thus from every actual tick the fixed 37 primitives generate a finite paid history when $A > 0$, or an exact zero-reserve disposition when $A = 0$, continuously through the same original Root. ■

Fixed source. MV3. Under `NoIslandNoMagic/Consciousness/Immortality/Embodied/Molecule/CPS1Personalized2025/`: `Source/NativeRegistration/{Source,Programme,Registration,Positive,BodyPositive}`, `Root/{NativeCurrent,Base,Installation}`, `Delivery/Runtime`, `Runtime/{RegisteredBody,BodyPaid,BodyFunction}`. Under `Lean/DomainProofs/CPS1ResourceExecution/CPS1MaterialIncidence/`: `NativeRoot{FromInput,Data,MaterialData,NextLaw}`, `Native{RawRegularity,FullSmooth,BodyPaidSource,BodyFunction}`, all `.lean`. The clinical label is the original source-row label; this proof derives no clinical effect from it and retains original Native.next separately from later governed action.

S6 Selection numbers and proof locations

The table matches claim numbers of the first-release selection to results of the main text and to the written proofs. The selection numbers agree with `core.Cn` in the first-release map of the public code selection (H0mework `docs/first-release-map.json`), so that each item can be checked.

Selection number	Result in the main text	Written proof	Source commit
core.C1	§6.1 Proposition 6.1	main text	H (base regression repair)
core.C2	§6.4 Definition 6.6, Theorem 6.7, Remark 6.8	main text	H
core.C3	§2.1 Definition 2.1, Proposition 2.2	main text	H
core.C4	§8.4 run period	main text	H
core.C5	§8.4 Proposition 8.9, Table 4, Figure 5	main text; Appendix C.3	H
core.C6	§8.4 Proposition 8.10	main text	H
core.C7	§3.1–§3.3 Definitions 3.1–3.2, Propositions 3.3–3.6	main text	H
core.C8	§4 Theorems 4.4, 4.6, 4.7, Proposition 4.8	main text; Appendices A–B	H
core.C9	§2.2–§2.4 Proposition 2.3, Definition 2.4, Proposition 2.5	main text; Appendix A.1	H

Selection number	Result in the main text	Written proof	Source commit
core.C10	§5 Definition 5.1, Proposition 5.2, Theorem 5.3	main text	H
core.C11	§7.1–§7.3 Proposition 7.1, Definition 7.2, Propositions 7.3–7.5	main text; Appendix C.6	H
core.C12	§6.2–§6.3 Definitions 6.2–6.3, Propositions 6.4–6.5, Table 2	main text; Appendix C.5	H
core.C13	§8.1–§8.3 Definition 8.1, Propositions 8.2–8.3, Theorem 8.4, Definition 8.5, Theorems 8.6–8.7, Proposition 8.8	main text; Appendices C.1–C.4	H
core.C22	§6.5 Definition 6.9, Theorem 6.10	S1	AB
core.C23	§3.4 Proposition 3.7, Definition 3.8, Theorem 3.9; §6.5 Proposition 6.11; §10.1	main text (complete)	AB
core.C24	§8.5 Definition 8.11, Theorem 8.12, Example 8.13, Figure 6	S2.1	AB
core.C25	§7.3 Proposition 7.6	main text (complete)	AB
core.C26	§7.3 Theorem 7.7(1)(2)	S2.2	AC
core.C27	§7.3 Theorem 7.7(3)	S2.2	AC
core.C28	§9.1 Definition 9.1, Theorem 9.2	S3.1	AD
core.C29	§9.2 Theorem 9.3	S3.2	AD
core.C30	§9.3 Definition 9.4, Theorem 9.5	S4.1	AD
core.C31	§9.3 Theorem 9.6	S4.2	AE
core.C32	§9.3 Theorem 9.7	S4.3	AE
core.C33	§9.4 Theorem 9.8(1)	S5.1	AE
core.C34	§9.4 Theorem 9.8(2)(3)	S5.1	AE

The four second-edition intake groups retain their respective fixed sources. The following table adds their written proof locations. CPS1 is an installation example in this edition, preserving the four intake groups.

Selection claim/example	Main result	Written proof	Source
C35–C36	§8.5 same environment and two charges; Theorem 8.14	S2.3	PR, H0-S
C37–C38	Theorem 8.14	S2.3	PR, MF
C39	Theorem 6.12; §7.3	S1.3	MV2
C40	Theorem 9.9	S4.4	MV2
C41	§10.1 original comb calculation	S5.5	MV2

Selection claim/example	Main result	Written proof	Source
C42	Theorem 8.15; §10.1 actual comb feedback	S5.2, S5.5	MV3
C43	Theorem 9.10; original compiler consumer of Theorem 6.12	S5.3	MV3
C44	Theorem 9.11	S3.3	MV3
C45	Theorem 9.12	S5.4	MV3
CPS1 installation	§10.1 37 primitives/body at every tick/finite paid history	S5.6	MV3